

FASTAPI COMPLETE MASTERY GUIDE

From Zero to Production-Ready in 7 Days

TABLE OF CONTENTS

1. [Quick Start Overview](#)
 2. [7-Day Learning Plan](#)
 3. [Part 1: Conceptual Foundation](#)
 4. [Part 2: Day 1 - The Fundamentals](#)
 5. [Part 3: Days 2-3 - Core Concepts Deep Dive](#)
 6. [Part 4: Days 4-5 - Advanced Features](#)
 7. [Part 5: Days 6-7 - Production & Real-World](#)
 8. [Common Mistakes & Misconceptions](#)
 9. [Advanced Concepts](#)
 10. [Resources & References](#)
 11. [Quick Revision Sheet](#)
 12. [Interview Questions](#)
-

QUICK START OVERVIEW

What is FastAPI?

FastAPI is a modern, high-performance Python web framework for building APIs (Application Programming Interfaces) with minimal code, automatic documentation, and built-in data validation.

Key Characteristics:

- **Fast:** 2-3x faster than Flask, comparable to Node.js and Go
- **Easy:** Intuitive syntax, minimal boilerplate
- **Automatic:** Self-generating API documentation (Swagger UI & ReDoc)
- **Robust:** Built-in data validation using Pydantic
- **Modern:** Uses Python 3.7+ async/await features
- **Production-Ready:** Used by companies like Uber, Microsoft, Netflix

Why FastAPI vs. Other Frameworks?

Feature	FastAPI	Flask	Django	Django REST
Performance	⚡ ⚡ ⚡ Very Fast	⚡ Moderate	⚡ ⚡ Good	⚡ ⚡ Good
Learning Curve	Easy	Easy	Medium	Medium
Auto Docs	✅ Built-in	❌ Manual	❌ Manual	❌ Manual
Data Validation	✅ Pydantic	❌ Manual	✅ ORM	✅ Serializers
Async Support	✅ Native	⚠️ Limited	⚠️ Limited	⚠️ Limited
Best For	APIs, Microservices	Lightweight APIs	Full-stack Web Apps	Large APIs

7-DAY LEARNING PLAN

Daily Breakdown

DAY 1: Foundation & Setup (3-4 hours)

Goal: Install, run first API, understand request-response cycle

- ✅ Install Python 3.9+, create virtual environment
- ✅ Install FastAPI & Uvicorn
- ✅ Create & run your first "Hello World" API
- ✅ Understand basic request/response flow
- ✅ Learn routing basics (GET, POST)
- ✅ Explore auto-generated documentation

Deliverable: Working API with 2 endpoints (GET, POST)

DAY 2: Core Concepts (4-5 hours)

Goal: Master path parameters, query parameters, and request body handling

- ✅ Path parameters (dynamic URLs)
- ✅ Query parameters (filtering & pagination)
- ✅ Request body with Pydantic models
- ✅ Response models and serialization
- ✅ Status codes and error handling
- ✅ Data validation and type hints

Deliverable: API with complex data validation and multiple parameter types

DAY 3: Intermediate Concepts (4-5 hours)

Goal: Deep understanding of Pydantic, validation, and request/response handling

- ☒ Advanced Pydantic models
- ☒ Nested models and relationships
- ☒ Custom validators
- ☒ Response model filtering
- ☒ Headers and cookies
- ☒ Form data handling

Deliverable: Full-featured CRUD API with validation

DAY 4: Advanced Features (4-5 hours)

Goal: Learn dependency injection, middleware, and security basics

- ☒ Dependency Injection system (Depends)
- ☒ Scopes and Dependency caching
- ☒ Middleware basics
- ☒ Authentication (Bearer tokens)
- ☒ OAuth2 & JWT basics
- ☒ CORS configuration

Deliverable: API with authentication and protected endpoints

DAY 5: Security & Performance (4-5 hours)

Goal: Master security patterns and optimize performance

- ☒ JWT token implementation
- ☒ Password hashing (bcrypt)
- ☒ Role-based access control
- ☒ Rate limiting and throttling
- ☒ Async operations and background tasks
- ☒ Performance optimization

Deliverable: Secure API with role-based access and background jobs

DAY 6: Database Integration (4-5 hours)

Goal: Connect FastAPI to databases using SQLAlchemy

- ☒ SQLAlchemy ORM basics
- ☒ Database models and migrations
- ☒ Session management
- ☒ Database CRUD operations
- ☒ Relationships (One-to-Many, Many-to-Many)
- ☒ Query optimization

Deliverable: Full API with database persistence

DAY 7: Production & Deployment (4-5 hours)

Goal: Deploy production-ready applications

- ☒ Environment configuration
- ☒ Error handling and logging
- ☒ API versioning
- ☒ Testing strategies
- ☒ Docker containerization
- ☒ Deployment options

Deliverable: Tested, containerized API ready for deployment

PART 1: CONCEPTUAL FOUNDATION

1.1 Understanding Web APIs

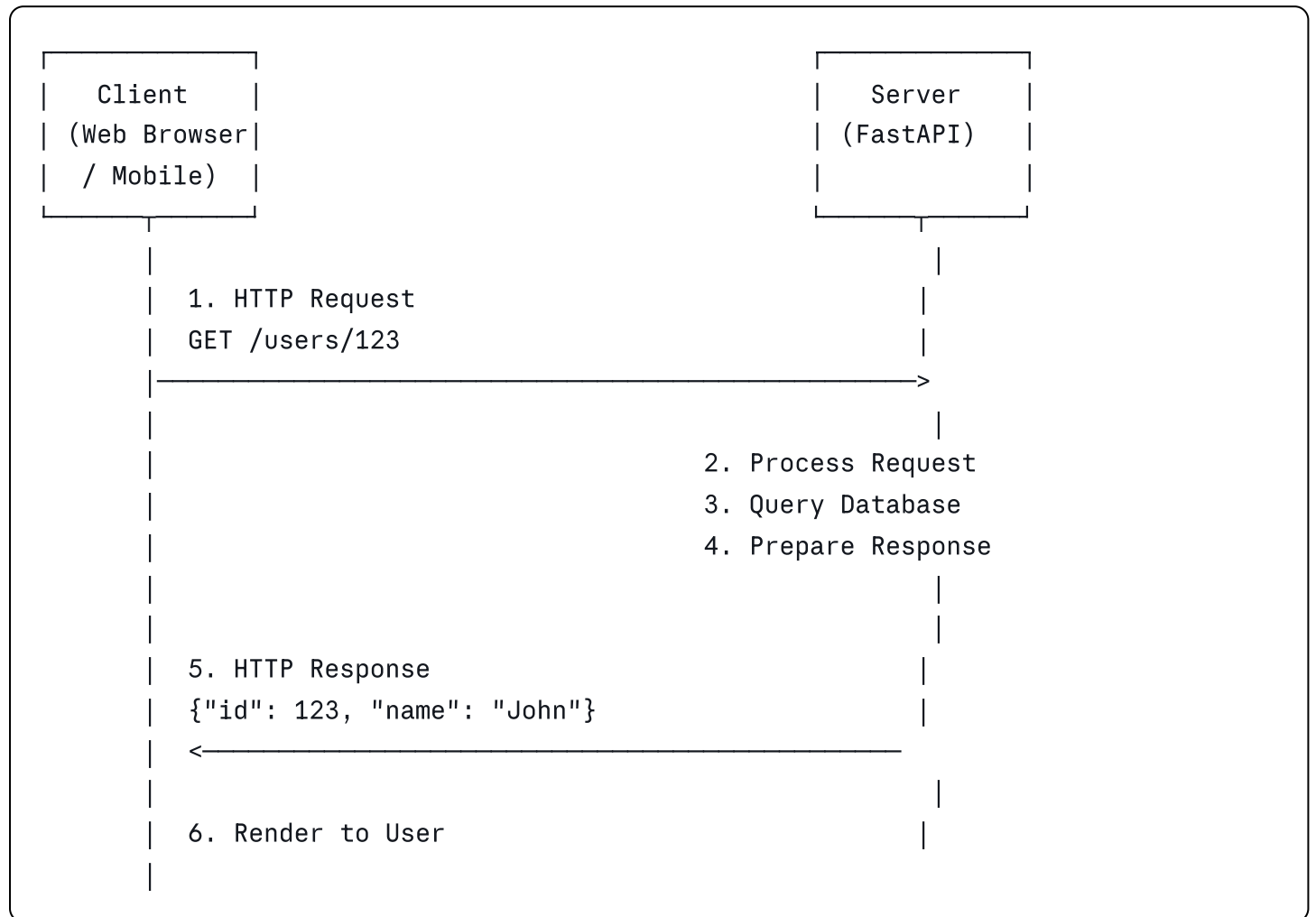
What is an API?

An API (Application Programming Interface) is a contract between a client and server that defines:

- What requests the client can make
- What responses the server will provide
- What data format is used

Real-World Analogy: Think of it like a restaurant:

- **Client** = Customer ordering food
- **API** = Menu and ordering system
- **Server** = Kitchen
- **Request** = Order placed
- **Response** = Food delivered



1.2 HTTP Basics for FastAPI

FastAPI is built on HTTP (HyperText Transfer Protocol). Understanding these is crucial:

HTTP Methods (Verbs):

Method	Purpose	Example
GET	Retrieve data	Get user profile
POST	Create new data	Create new user
PUT	Update entire resource	Replace user data
PATCH	Partial update	Update user email only
DELETE	Remove data	Delete user account

HTTP Status Codes:

Code	Meaning	Example
200	OK	Request successful
201	Created	New resource created
400	Bad Request	Invalid input
401	Unauthorized	Missing authentication
403	Forbidden	Authenticated but no permission
404	Not Found	Resource doesn't exist
500	Server Error	Server-side problem

1.3 Why FastAPI is Revolutionary

Before FastAPI (Flask):

```
python
```

```
# You had to do everything manually
@app.route('/users/<int:user_id>')
def get_user(user_id):
    # Manual validation
    if not isinstance(user_id, int):
        return {"error": "Invalid ID"}, 400

    # Manual serialization
    user = db.query(User).get(user_id)
    return {
        "id": user.id,
        "name": user.name,
        "email": user.email
    }
```

With FastAPI:

```
python

# Automatic validation, serialization, documentation
@app.get("/users/{user_id}", response_model=UserResponse)
async def get_user(user_id: int):
    user = db.query(User).get(user_id)
    return user
# Everything else is automatic!
```

Key Advantages:

1. **Type hints** → Automatic validation + documentation
2. **Pydantic models** → Built-in serialization & validation
3. **Async support** → Handle thousands of concurrent requests
4. **Auto-generated docs** → Swagger UI included for free
5. **Less code** → 40-50% less boilerplate than Flask

1.4 Understanding the Request-Response Cycle

FastAPI REQUEST CYCLE

1. CLIENT SENDS REQUEST

GET /users/123?skip=0&limit=10

Headers: {Authorization: "Bearer TOKEN"}

Body: (none for GET)

2. UVICORN (ASGI SERVER) RECEIVES REQUEST

- Parses HTTP request
- Routes to appropriate handler

3. FASTAPI PROCESSES REQUEST

- Extract path parameter: `user_id = 123`
- Extract query params: `skip=0, limit=10`
- Extract headers: `token = "TOKEN"`
- Validate types: `user_id` must be `int` ✓
- Run dependencies (auth, etc.)
- Call route handler function

4. ROUTE HANDLER EXECUTES

```
def get_users(user_id: int, skip: int, limit: int):  
    # Your business logic here  
    users = db.query(User).offset(skip).limit(limit)  
    return users
```

5. FASTAPI SERIALIZES RESPONSE

- Convert Python objects to JSON
- Use `response_model` for validation
- Apply filters/inclusions

6. SERVER SENDS RESPONSE

Status: 200 OK

Headers: {Content-Type: "application/json"}

Body: [{"id": 123, "name": "John"}, ...]

7. CLIENT RECEIVES & PROCESSES

- Parses JSON response
- Updates UI or processes data

PART 2: DAY 1 - THE FUNDAMENTALS

2.1 Installation & Setup

Step 1: System Requirements

- Python 3.7+ (recommended: 3.9 or 3.10)
- pip (Python package manager)
- Virtual environment (recommended)

Step 2: Create Virtual Environment

```
bash

# Create directory
mkdir fastapi_project
cd fastapi_project

# Create virtual environment
python -m venv venv

# Activate virtual environment
# On macOS/Linux:
source venv/bin/activate

# On Windows:
venv\Scripts\activate
```

Step 3: Install Dependencies

```
bash

# Install FastAPI
pip install fastapi

# Install Uvicorn (ASGI server)
pip install uvicorn[standard]

# Optional but recommended
pip install python-multipart # For form data
pip install email-validator  # For email validation
```

Step 4: Verify Installation

```
bash

python -c "import fastapi; print(fastapi.__version__)"
python -c "import uvicorn; print(uvicorn.__version__)"
```

2.2 Your First API

Code: `main.py`

```
python

# Import FastAPI
from fastapi import FastAPI

# Create application instance
app = FastAPI(
    title="My First API",
    description="A simple API to learn FastAPI",
    version="1.0.0"
)

# Define first endpoint (GET request)
@app.get("/")
def read_root():
    """
    Root endpoint that returns a welcome message.
    """
    return {"message": "Welcome to FastAPI!"}

# Define second endpoint with path parameter
@app.get("/greet/{name}")
def greet_user(name: str):
    """
    Greet a specific user by name.

    Args:
        name: The name of the person to greet

    Returns:
        A greeting message
    """
    return {"greeting": f"Hello, {name}!"}

# Run server: uvicorn main:app --reload
```

How It Works

```
Decorator: @app.get("/")
           ↑
           Tells FastAPI this function handles GET requests to "/"

Function:  def read_root():
           ↑
           The actual handler that processes the request

Return:    {"message": "..."}
           ↑
           Automatically converted to JSON response
```

Running Your API

```
bash

# Start development server
uvicorn main:app --reload

# Output:
# INFO:      Uvicorn running on http://127.0.0.1:8000
# INFO:      Application startup complete
```

Test Your API

Option 1: Browser

- Visit: <http://localhost:8000>
- Visit: <http://localhost:8000/greet/Alice>

Option 2: Using curl

```
bash

curl http://localhost:8000
curl http://localhost:8000/greet/Alice
```

Option 3: Python requests

```
python
```

```
import requests

response = requests.get("http://localhost:8000")
print(response.json()) # {'message': 'Welcome to FastAPI!'}

response = requests.get("http://localhost:8000/greet/Alice")
print(response.json()) # {'greeting': 'Hello, Alice!'}
```

2.3 Understanding Routing & Decorators

What is a Decorator?

A decorator (`@app.get()`) is Python syntax that modifies a function's behavior:

```
python

@app.get("/")          # Decorator: tells FastAPI to route GET "/" to this function
def read_root():       # This function is now a route handler
    return {"msg": "Hello"}
```

Deep Dive: How Decorators Work

```
python

# Without decorator (pseudo-code showing what happens)
def read_root():
    return {"msg": "Hello"}

# Decorator is equivalent to:
read_root = app.get("/")(read_root)
# Now read_root is wrapped and registered with FastAPI

# When request comes to GET "/", FastAPI:
# 1. Finds the registered function
# 2. Calls it with appropriate parameters
# 3. Returns the result as JSON
```

HTTP Methods in FastAPI

```
python
```

```
from fastapi import FastAPI

app = FastAPI()

# GET: Retrieve data
@app.get("/items")
def list_items():
    return {"items": []}

# POST: Create new data
@app.post("/items")
def create_item(item: dict):
    return {"created": item}

# PUT: Replace entire resource
@app.put("/items/{id}")
def update_item(id: int, item: dict):
    return {"updated": item}

# PATCH: Partial update
@app.patch("/items/{id}")
def patch_item(id: int, item: dict):
    return {"patched": item}

# DELETE: Remove resource
@app.delete("/items/{id}")
def delete_item(id: int):
    return {"deleted": id}
```

2.4 Auto-Generated Documentation

One of FastAPI's superpowers: **automatic API documentation**.

Interactive API Documentation

Swagger UI:

<http://localhost:8000/docs>

ReDoc (Alternative UI):

<http://localhost:8000/redoc>

OpenAPI Schema (JSON):

```
http://localhost:8000/openapi.json
```

How Documentation is Generated

FastAPI automatically generates docs from:

1. **Type hints** in function parameters
2. **Docstrings** in your functions
3. **Decorator information** (@app.get, etc.)

```
python

@app.get("/users/{user_id}")
def get_user(user_id: int):
    """
    Get a specific user by their ID.

    Parameters:
        user_id: The unique identifier of the user

    Returns:
        User information as JSON
    """
    return {"user_id": user_id}

# This automatically appears in Swagger UI docs!
```

2.5 Understanding Path Parameters

Path parameters are dynamic parts of the URL:

```
URL: /users/123
    |_____|
    |_____| Path: "users"
           Parameter: "123"
```

Type Validation

FastAPI validates path parameters based on type hints:

```
python
```

```
@app.get("/users/{user_id}")
def get_user(user_id: int): # Must be integer
    return {"user_id": user_id}

# Valid: GET /users/123 ✓
# Invalid: GET /users/abc → 422 Validation Error
```

Path Parameter Order Matters

```
python

# Define specific routes BEFORE generic ones:

@app.get("/users/me")
def get_current_user():
    return {"user": "current_user"}

@app.get("/users/{user_id}")
def get_user(user_id: int):
    return {"user_id": user_id}

# GET /users/me → Returns current_user (specific route matched first)
# GET /users/123 → Returns user_id (generic route)
```

2.6 Query Parameters

Query parameters come after `(?)` in the URL:

URL: /search?query=python&limit=10

Query parameters

Basic Query Parameters

```

@app.get("/search")
def search(query: str, limit: int = 10):
    """
    Search endpoint.

    Args:
        query: Search term (required)
        limit: Maximum results (optional, default=10)
    """
    return {
        "query": query,
        "limit": limit,
        "results": [f"Result {i}" for i in range(limit)]
    }

# Usage:
# GET /search?query=python → limit defaults to 10
# GET /search?query=python&limit=5 → limit is 5
# GET /search → Error! query is required (no default)

```

Optional Query Parameters

```

python

from typing import Optional

@app.get("/items")
def list_items(skip: int = 0, limit: Optional[int] = None):
    """
    List items with optional pagination.

    Args:
        skip: Number of items to skip (default=0)
        limit: Maximum items to return (optional)
    """
    items = ["item1", "item2", "item3", "item4", "item5"]

    items = items[skip:]
    if limit:
        items = items[:limit]

    return {"items": items}

# GET /items → All items
# GET /items?skip=1&limit=2 → Skip 1, return max 2

```


2.7 Request Body & POST Requests

Request body is data sent in the request (typically JSON):

```
POST /users
{
  "name": "John",
  "email": "john@example.com",
  "age": 30
}
```

Using Pydantic Models for Validation

```
python

from pydantic import BaseModel

# Define what data structure we expect
class User(BaseModel):
    name: str
    email: str
    age: int

@app.post("/users")
def create_user(user: User):
    """
    Create a new user.

    The 'user' parameter automatically:
    - Parses JSON from request body
    - Validates all fields (names, types)
    - Converts to Python object
    """
    return {
        "message": f"User {user.name} created",
        "user": user
    }
```

How It Works

1. Client sends JSON:
POST /users
{ "name": "John", "email": "john@example.com", "age": 30 }
2. FastAPI:
 - Reads JSON from request body

- Validates against User model
- Converts to User object
- Passes to function

3. Your function receives:

```
user = User(  
    name="John",  
    email="john@example.com",  
    age=30  
)
```

4. Return Python dict/object:

- FastAPI converts back to JSON
- Sends to client

Automatic Validation

python

```
class User(BaseModel):  
    name: str  
    email: str  
    age: int  
  
# POST /users  
# {"name": "John", "email": "john"} # Missing 'age'  
# → 422 Validation Error  
#  
# {"name": "John", "email": "john@example.com", "age": "thirty"}  
# → 422 Validation Error (age must be int)
```

2.8 Status Codes & Responses

Custom Status Codes

python

```

from fastapi import FastAPI, status

app = FastAPI()

@app.post("/users", status_code=status.HTTP_201_CREATED)
def create_user(user: dict):
    """Returns 201 Created instead of default 200"""
    return {"created": True, "user": user}

@app.delete("/users/{user_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_user(user_id: int):
    """Returns 204 No Content"""
    return None

@app.get("/users/{user_id}", status_code=status.HTTP_200_OK)
def get_user(user_id: int):
    """Returns 200 OK (default for GET)"""
    return {"user_id": user_id}

```

2.9 DAY 1 EXERCISES

Exercise 1.1: Create a Basic Todo API

Objective: Build a simple Todo API without database

```

python

# Create a main.py with these endpoints:

# 1. GET /todos - Return all todos
#   Response: {"todos": [{"id": 1, "title": "Learn FastAPI"}]}

# 2. GET /todos/{todo_id} - Get specific todo
#   Response: {"id": 1, "title": "Learn FastAPI"}

# 3. POST /todos - Create new todo
#   Request body: {"title": "New Task", "description": "Details"}
#   Response: 201 Created with the new todo

# 4. DELETE /todos/{todo_id} - Delete todo
#   Response: {"deleted": true}

```

Solution Outline:

```

python

```

```

from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class TodoCreate(BaseModel):
    title: str
    description: str = ""

# In-memory storage (not for production!)
todos_db = [
    {"id": 1, "title": "Learn FastAPI", "completed": False}
]

@app.get("/todos")
def list_todos():
    return {"todos": todos_db}

@app.get("/todos/{todo_id}")
def get_todo(todo_id: int):
    for todo in todos_db:
        if todo["id"] == todo_id:
            return todo
    return {"error": "Not found"}

@app.post("/todos", status_code=201)
def create_todo(todo: TodoCreate):
    new_todo = {
        "id": len(todos_db) + 1,
        "title": todo.title,
        "description": todo.description,
        "completed": False
    }
    todos_db.append(new_todo)
    return new_todo

@app.delete("/todos/{todo_id}")
def delete_todo(todo_id: int):
    global todos_db
    todos_db = [t for t in todos_db if t["id"] != todo_id]
    return {"deleted": True}

```

Exercise 1.2: Calculator API

Objective: Create an API that performs basic math operations

```
python
```

```
# Endpoints:
# GET /add?a=5&b=3 → {"result": 8}
# GET /subtract?a=10&b=3 → {"result": 7}
# GET /multiply?a=4&b=5 → {"result": 20}
# GET /divide?a=20&b=4 → {"result": 5.0}
# GET /divide?a=20&b=0 → Error: Division by zero
```

Exercise 1.3: Product Catalog

Objective: Create a product listing API

```
python
```

```
# Endpoints:
# GET /products - List all products
# GET /products/{product_id} - Get specific product
# GET /products?category=electronics - Filter by category
# POST /products - Add new product
#
# Use Pydantic models for data validation
```

PART 3: DAYS 2-3 - CORE CONCEPTS DEEP DIVE

3.1 Deep Dive: Pydantic Models

What is Pydantic?

Pydantic is a Python library that enforces type hints at runtime. It's used by FastAPI for:

1. **Validation** - Ensure data is correct type/format
2. **Serialization** - Convert Python objects to JSON
3. **Documentation** - Automatic API docs from models

Basic Model Definition

```
python
```

```
from pydantic import BaseModel

class User(BaseModel):
    id: int
    name: str
    email: str
    age: int
    is_active: bool = True # Default value

# Creating instance
user = User(
    id=1,
    name="John",
    email="john@example.com",
    age=30
)

# Accessing attributes
print(user.name) # "John"
print(user.is_active) # True

# Convert to dict
user_dict = user.dict()
# {'id': 1, 'name': 'John', 'email': 'john@example.com', 'age': 30, 'is_active': True}

# Convert to JSON
user_json = user.json()
# '{"id": 1, "name": "John", ...}'
```

Type Validation

python

```

from pydantic import BaseModel

class User(BaseModel):
    name: str
    age: int

# Valid: Matches types
user = User(name="John", age=30) ✓

# Invalid: String instead of int
user = User(name="John", age="thirty")
# → ValidationError!

# Invalid: Missing required field
user = User(name="John")
# → ValidationError!

# Type coercion: Pydantic tries to convert
user = User(name="John", age="30")
# ✓ Works! Converts string "30" to int 30

```

Optional Fields

```

python

from typing import Optional
from pydantic import BaseModel

class User(BaseModel):
    name: str
    email: str
    phone: Optional[str] = None # Can be null
    age: int = 18 # Has default

# Valid: phone is optional
user = User(name="John", email="john@example.com")
# phone defaults to None, age defaults to 18

# Valid: explicit phone
user = User(name="John", email="john@example.com", phone="555-1234")

# Valid: explicit None
user = User(name="John", email="john@example.com", phone=None)

```

Nested Models

```
python

from pydantic import BaseModel

class Address(BaseModel):
    street: str
    city: str
    country: str

class User(BaseModel):
    name: str
    email: str
    address: Address # Nested model

# Usage
user = User(
    name="John",
    email="john@example.com",
    address={
        "street": "123 Main St",
        "city": "New York",
        "country": "USA"
    }
)

# Access nested data
print(user.address.city) # "New York"

# Automatic validation of nested model
user = User(
    name="John",
    email="john@example.com",
    address={
        "street": "123 Main St"
        # Missing city and country
    }
)
# → ValidationError!
```

Lists and Collections

```
python
```



```
from typing import List
from pydantic import BaseModel

class Address(BaseModel):
    street: str
    city: str

class User(BaseModel):
    name: str
    addresses: List[Address] # List of models
    tags: List[str] # List of strings

# Usage
user = User(
    name="John",
    addresses=[
        {"street": "123 Main", "city": "NYC"},
        {"street": "456 Oak", "city": "LA"}
    ],
    tags=["developer", "python"]
)

# Access
print(user.addresses[0].city) # "NYC"
print(user.tags) # ["developer", "python"]
```

Custom Validators

python

```

from pydantic import BaseModel, validator, ValidationError

class User(BaseModel):
    name: str
    email: str
    age: int

    @validator('name')
    def name_must_not_be_empty(cls, v):
        """Validate that name is not empty"""
        if not v.strip():
            raise ValueError('Name cannot be empty')
        return v.strip()

    @validator('email')
    def email_must_contain_at(cls, v):
        """Validate email format"""
        if '@' not in v:
            raise ValueError('Invalid email format')
        return v.lower()

    @validator('age')
    def age_must_be_positive(cls, v):
        """Validate age is positive"""
        if v < 0:
            raise ValueError('Age cannot be negative')
        if v > 150:
            raise ValueError('Age seems unrealistic')
        return v

# Test validation
try:
    user = User(name="", email="john@example.com", age=25)
except ValidationError as e:
    print(e) # name validation error

```

Root Validators

python

```
from pydantic import BaseModel, root_validator

class UserRegistration(BaseModel):
    password: str
    password_confirm: str

    @root_validator
    def passwords_match(cls, values):
        """Validate that passwords match"""
        password = values.get('password')
        password_confirm = values.get('password_confirm')

        if password != password_confirm:
            raise ValueError('Passwords do not match')
        return values

# Test
try:
    user = UserRegistration(
        password="abc123",
        password_confirm="abc124"  # Doesn't match
    )
except ValueError:
    print("Passwords don't match")
```

3.2 Response Models & Serialization

Using response_model

```
python
```

```

from fastapi import FastAPI
from pydantic import BaseModel

class User(BaseModel):
    id: int
    name: str
    email: str
    is_admin: bool = False

app = FastAPI()

@app.get("/users/{user_id}", response_model=User)
def get_user(user_id: int):
    """
    Response will be validated against User model.
    Only fields in User model will be returned.
    """
    return {
        "id": user_id,
        "name": "John",
        "email": "john@example.com",
        "is_admin": False,
        "password_hash": "secret123" # Not in model, will be excluded!
    }

# Response to client:
# {
#     "id": 1,
#     "name": "John",
#     "email": "john@example.com",
#     "is_admin": false
# }
# Note: password_hash is NOT returned!

```

Why response_model is Important

SECURITY: Hide sensitive fields

Database object has:	
- id, name, email	
- password_hash ← Secret!	
- phone_backup ← Private!	
response_model User shows:	
- id, name, email only ✓	

DOCUMENTATION: API docs show exactly what client receives

```
| Swagger UI displays: |  
| { |  
|   "id": 0, |  
|   "name": "string", |  
|   "email": "string" |  
| } |
```

Response Model List

```
python  
  
from typing import List  
from fastapi import FastAPI  
from pydantic import BaseModel  
  
class UserResponse(BaseModel):  
    id: int  
    name: str  
    email: str  
  
app = FastAPI()  
  
@app.get("/users", response_model=List[UserResponse])  
def list_users():  
    """Returns list of users"""  
    return [  
        {"id": 1, "name": "John", "email": "john@example.com"},  
        {"id": 2, "name": "Jane", "email": "jane@example.com"}  
    ]  
  
# Client receives:  
# [  
#     {"id": 1, "name": "John", "email": "john@example.com"},  
#     {"id": 2, "name": "Jane", "email": "jane@example.com"}  
# ]
```

3.3 Handling Form Data

Form Data vs JSON

Form Data (application/x-www-form-urlencoded):

```
name=John&age=30&city=NYC
```

JSON Data (application/json):

```
{  
    "name": "John",  
    "age": 30,  
    "city": "NYC"  
}
```

Receiving Form Data

python

```
from fastapi import FastAPI, Form  
from pydantic import BaseModel  
  
app = FastAPI()  
  
# Simple form submission  
@app.post("/login")  
def login(username: str = Form(...), password: str = Form(...)):  
    """  
    Form(...) tells FastAPI this comes from form, not JSON body.  
    ... means it's required.  
    """  
    return {"username": username}  
  
# HTML form:  
# <form action="/login" method="post">  
#     <input name="username" />  
#     <input name="password" type="password" />  
#     <button type="submit">Login</button>  
# </form>
```

Mixing Form and JSON

python

```
from fastapi import FastAPI, Form, File, UploadFile
from pydantic import BaseModel

app = FastAPI()

class UserData(BaseModel):
    name: str
    email: str

@app.post("/profile")
def create_profile(
    user_data: UserData, # JSON body
    favorite_color: str = Form(...) # Form field
):
    """Combine JSON body with form fields"""
    return {
        "user": user_data,
        "color": favorite_color
    }
```

3.4 File Uploads

Simple File Upload

```
python
```

```

from fastapi import FastAPI, UploadFile, File

app = FastAPI()

@app.post("/upload")
async def upload_file(file: UploadFile = File(...)):
    """
    UploadFile is a special type for file uploads.
    Provides: filename, content_type, file (async file object)
    """
    contents = await file.read()

    return {
        "filename": file.filename,
        "size": len(contents),
        "content_type": file.content_type
    }

# Client sends multipart/form-data:
# file: [binary data]

```

Multiple File Upload

```

python

from typing import List
from fastapi import FastAPI, UploadFile, File

app = FastAPI()

@app.post("/upload-multiple")
async def upload_multiple(files: List[UploadFile] = File(...)):
    """Upload multiple files"""
    results = []

    for file in files:
        contents = await file.read()
        results.append({
            "filename": file.filename,
            "size": len(contents)
        })

    return results

```


Saving Files

```
python

import os
from fastapi import FastAPI, UploadFile, File

app = FastAPI()

@app.post("/upload")
async def upload_and_save(file: UploadFile = File(...)):
    """Upload and save file to disk"""
    # Create uploads directory if not exists
    os.makedirs("uploads", exist_ok=True)

    # Save file
    file_path = f"uploads/{file.filename}"
    with open(file_path, "wb") as f:
        contents = await file.read()
        f.write(contents)

    return {
        "filename": file.filename,
        "path": file_path
    }
```

3.5 Headers and Cookies

Reading Request Headers

```
python
```

```

from fastapi import FastAPI, Header

app = FastAPI()

@app.get("/items")
def read_items(
    x_custom_header: str = Header(...),
    user_agent: str = Header(...)
):
    """
    Header(...) extracts from HTTP headers.
    FastAPI automatically converts header names:
    x_custom_header → X-Custom-Header
    user_agent → User-Agent
    """
    return {
        "custom": x_custom_header,
        "user_agent": user_agent
    }

# Client sends:
# GET /items HTTP/1.1
# X-Custom-Header: custom-value
# User-Agent: Mozilla/5.0

```

Setting Response Headers

```

python

from fastapi import FastAPI

app = FastAPI()

@app.get("/items")
def read_items(response: Response):
    """Set headers in response"""
    response.headers["X-Custom-Header"] = "custom-value"
    response.headers["X-Total-Count"] = "100"

    return {"items": []}

```

Reading Cookies

```

python

```

```
from fastapi import FastAPI, Cookie

app = FastAPI()

@app.get("/items")
def read_items(session_id: str = Cookie(...)):
    """Read cookie value"""
    return {"session_id": session_id}

# Client sends:
# GET /items HTTP/1.1
# Cookie: session_id=abc123
```

Setting Cookies

```
python

from fastapi import FastAPI
from fastapi.responses import Response

app = FastAPI()

@app.post("/login")
def login(response: Response):
    """Set cookie in response"""
    response.set_cookie(
        key="session_id",
        value="abc123",
        max_age=3600, # 1 hour
        httponly=True, # Not accessible by JavaScript
        secure=True, # Only over HTTPS
        samesite="strict" # CSRF protection
    )

    return {"logged_in": True}
```

3.6 Error Handling & Status Codes

HTTPException

```
python
```

```
from fastapi import FastAPI, HTTPException, status

app = FastAPI()

users_db = {1: "John", 2: "Jane"}

@app.get("/users/{user_id}")
def get_user(user_id: int):
    if user_id not in users_db:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="User not found",
            headers={"X-Error": "custom-error-header"}
        )

    return {"name": users_db[user_id]}

# GET /users/999:
# Response: 404 Not Found
# {
#     "detail": "User not found"
# }
```

Custom Exception Handlers

python

```

from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse

class CustomException(Exception):
    def __init__(self, message: str):
        self.message = message

app = FastAPI()

@app.exception_handler(CustomException)
async def custom_exception_handler(request: Request, exc: CustomException):
    """Handle custom exceptions"""
    return JSONResponse(
        status_code=400,
        content={"detail": exc.message}
    )

@app.get("/items/{item_id}")
def read_item(item_id: int):
    if item_id == 0:
        raise CustomException("Item ID cannot be zero")
    return {"item_id": item_id}

```

3.7 Complete CRUD API Example

python

```
from fastapi import FastAPI, HTTPException, status
from pydantic import BaseModel
from typing import List

app = FastAPI(title="Complete CRUD API")

# Models
class ItemCreate(BaseModel):
    name: str
    description: str = ""
    price: float

class ItemUpdate(BaseModel):
    name: str = None
    description: str = None
    price: float = None

class ItemResponse(BaseModel):
    id: int
    name: str
    description: str
    price: float

# In-memory database
items_db = {
    1: {"id": 1, "name": "Laptop", "description": "High-end laptop", "price": 1000},
    2: {"id": 2, "name": "Mouse", "description": "Wireless mouse", "price": 25}
}
next_id = 3

# CREATE
@app.post("/items", response_model=ItemResponse, status_code=status.HTTP_201_CREATED)
def create_item(item: ItemCreate):
    global next_id

    new_item = {
        "id": next_id,
        "name": item.name,
        "description": item.description,
        "price": item.price
    }

    items_db[next_id] = new_item
    next_id += 1

    return new_item
```

```
# READ (List)
@app.get("/items", response_model=List[ItemResponse])
def list_items(skip: int = 0, limit: int = 10):
    items = list(items_db.values())
    return items[skip:skip + limit]

# READ (Single)
@app.get("/items/{item_id}", response_model=ItemResponse)
def get_item(item_id: int):
    if item_id not in items_db:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="Item not found"
        )

    return items_db[item_id]

# UPDATE
@app.put("/items/{item_id}", response_model=ItemResponse)
def update_item(item_id: int, item: ItemUpdate):
    if item_id not in items_db:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="Item not found"
        )

    existing = items_db[item_id]
    update_data = item.dict(exclude_unset=True)
    updated = existing.copy()
    updated.update(update_data)

    items_db[item_id] = updated

    return updated

# DELETE
@app.delete("/items/{item_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_item(item_id: int):
    if item_id not in items_db:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="Item not found"
        )
```

```
del items_db[item_id]
return None
```

3.8 DAYS 2-3 EXERCISES

Exercise 3.1: Blog API

Create a blogging platform API with:

```
python

# Models:
class Post(BaseModel):
    id: int
    title: str
    content: str
    author: str
    published: bool = False

class Comment(BaseModel):
    id: int
    post_id: int
    author: str
    content: str

# Endpoints:
# POST /posts - Create post
# GET /posts - List posts (with pagination)
# GET /posts/{post_id} - Get post
# PUT /posts/{post_id} - Update post
# DELETE /posts/{post_id} - Delete post
# POST /posts/{post_id}/comments - Add comment
# GET /posts/{post_id}/comments - List comments
```

Exercise 3.2: E-commerce Product API

```
python
```



```
# Advanced features:
# - Product categories
# - Stock management
# - Price validation (must be > 0)
# - Product search/filtering
# - Inventory tracking

# GET /products?category=electronics&min_price=100&max_price=500
# GET /products/{product_id}/stock
# POST /products/{product_id}/stock/deduct
```

Exercise 3.3: User Registration System

```
python

# Features:
# - Email validation
# - Password validation (min 8 chars, complexity)
# - Custom validators
# - Password confirmation matching
# - Error handling for duplicate emails

class UserRegistration(BaseModel):
    email: str
    password: str
    password_confirm: str
    first_name: str
    last_name: str
```

PART 4: DAYS 4-5 - ADVANCED FEATURES

4.1 Dependency Injection System

The Problem It Solves

```
python
```

```

# Without Dependency Injection:
@app.get("/users")
def get_users():
    # How do we get database connection?
    # How do we authenticate user?
    # Code is tightly coupled and hard to test
    db = Database()
    user = get_current_user()
    return db.query(User)

# With Dependency Injection:
@app.get("/users")
def get_users(db: Database = Depends(get_database)):
    # Database is injected
    # Clean separation of concerns
    # Easy to test with mock objects
    return db.query(User)

```

Basic Dependency

```

python

from fastapi import FastAPI, Depends

app = FastAPI()

# Define a dependency function
def get_query_param(q: str = ""):
    """A simple dependency that extracts query parameter"""
    return q

@app.get("/search")
def search(query: str = Depends(get_query_param)):
    """query parameter comes from dependency"""
    return {"query": query}

# GET /search?q=python
# → query = "python"

```

Dependency with Business Logic

```

python

```

```

from fastapi import FastAPI, Depends, HTTPException

app = FastAPI()

# Simulated database
users_db = {
    "john": {"id": 1, "name": "John"},
    "jane": {"id": 2, "name": "Jane"}
}

def get_user_by_username(username: str):
    """Dependency: Get user from database"""
    if username not in users_db:
        raise HTTPException(status_code=404, detail="User not found")
    return users_db[username]

@app.get("/users/{username}")
def get_user(user: dict = Depends(get_user_by_username)):
    """
    Instead of:
        user = get_user_by_database(username)
        if not user:
            raise HTTPException(...)

    Just use the dependency!
    """
    return user

# GET /users/john
# → Returns {"id": 1, "name": "John"}
#
# GET /users/unknown
# → HTTPException raised automatically

```

Dependency with Database

python

```

from fastapi import FastAPI, Depends
from sqlalchemy import create_engine
from sqlalchemy.orm import Session, sessionmaker

app = FastAPI()

# Database setup
DATABASE_URL = "sqlite:///./test.db"
engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(bind=engine)

def get_db():
    """Dependency: Provide database session"""
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

@app.get("/users/{user_id}")
def get_user(user_id: int, db: Session = Depends(get_db)):
    """db session is automatically provided"""
    user = db.query(User).filter(User.id == user_id).first()
    return user

```

How it works:

```

Request comes in
    ↓
FastAPI sees: Depends(get_db)
    ↓
FastAPI calls get_db()
    ↓
get_db() creates database session
    ↓
Session passed to route handler
    ↓
Handler processes request
    ↓
Finally block runs, session closes

```

Dependency Scopes

python

```

from fastapi import FastAPI, Depends

app = FastAPI()

# Function scope (default)
def get_query():
    """Called every time it's needed"""
    return "query_value"

@app.get("/items")
def list_items(q: str = Depends(get_query)):
    # get_query() called for this request
    return {"q": q}

# Caching dependencies
from fastapi import FastAPI, Depends

class DatabaseConnection:
    def __init__(self):
        self.connected = False

    def connect(self):
        self.connected = True
        print("Connected to database")

def get_database():
    db = DatabaseConnection()
    db.connect()
    return db

@app.get("/items")
def list_items(db: DatabaseConnection = Depends(get_database, use_cache=True)):
    # In single request, same db instance reused
    return {"status": db.connected}

```

Nested Dependencies

python

```

from fastapi import FastAPI, Depends

app = FastAPI()

def get_token(authorization: str = Header(...)):
    """Extract token from header"""
    return authorization.replace("Bearer ", "")

def get_current_user(token: str = Depends(get_token)):
    """Get user from token (depends on get_token)"""
    if token == "secret123":
        return {"id": 1, "name": "John"}
    raise HTTPException(status_code=401)

@app.get("/users/me")
def get_me(user: dict = Depends(get_current_user)):
    """
    Dependency chain:
    get_me depends on get_current_user
    get_current_user depends on get_token

    FastAPI automatically resolves the chain!
    """
    return user

```

4.2 Authentication & Security

Basic Authentication

```
python
```

```

from fastapi import FastAPI, Depends, HTTPException, status
from fastapi.security import HTTPBasic, HTTPBasicCredentials

app = FastAPI()
security = HTTPBasic()

users_db = {
    "john": "password123",
    "jane": "pass456"
}

@app.get("/secure")
def secure_endpoint(credentials: HTTPBasicCredentials = Depends(security)):
    """Basic HTTP authentication"""

    if credentials.username not in users_db:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid credentials",
            headers={"WWW-Authenticate": "Basic"}
        )

    if users_db[credentials.username] != credentials.password:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid credentials",
            headers={"WWW-Authenticate": "Basic"}
        )

    return {"message": f"Hello {credentials.username}"}

# Client sends:
# Authorization: Basic am9objpwYXNzd29yZDEyMw==

```

Bearer Token Authentication

```
python
```

```
from fastapi import FastAPI, Depends, HTTPException, status
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials

app = FastAPI()
security = HTTPBearer()

def verify_token(credentials: HTTPAuthorizationCredentials = Depends(security)):
    """Verify bearer token"""
    token = credentials.credentials

    if token != "valid-token-123":
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid token"
        )

    return token

@app.get("/secure")
def secure_endpoint(token: str = Depends(verify_token)):
    return {"message": "Access granted", "token": token}

# Client sends:
# Authorization: Bearer valid-token-123
```

JWT (JSON Web Token) Implementation

python


```
from datetime import datetime, timedelta
from jose import JWTError, jwt
from passlib.context import CryptContext
from fastapi import FastAPI, Depends, HTTPException, status
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
from pydantic import BaseModel

# Configuration
SECRET_KEY = "your-secret-key-keep-it-safe"
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30

# Password hashing
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

# Models
class Token(BaseModel):
    access_token: str
    token_type: str

class TokenData(BaseModel):
    username: str = None

class User(BaseModel):
    username: str
    email: str = None
    full_name: str = None
    disabled: bool = False

class UserInDB(User):
    hashed_password: str

app = FastAPI()

# Simulated database
fake_users_db = {
    "john": {
        "username": "john",
        "full_name": "John Doe",
        "email": "john@example.com",
        "hashed_password": pwd_context.hash("password123"),
        "disabled": False
    }
}

def verify_password(plain_password, hashed_password):
```

```

    """Verify password against hash"""
    return pwd_context.verify(plain_password, hashed_password)

def get_password_hash(password):
    """Hash a password"""
    return pwd_context.hash(password)

def authenticate_user(username: str, password: str):
    """Authenticate user with username and password"""
    if username not in fake_users_db:
        return False

    user = fake_users_db[username]

    if not verify_password(password, user["hashed_password"]):
        return False

    return user

def create_access_token(data: dict, expires_delta: timedelta = None):
    """Create JWT token"""
    to_encode = data.copy()

    if expires_delta:
        expire = datetime.utcnow() + expires_delta
    else:
        expire = datetime.utcnow() + timedelta(minutes=15)

    to_encode.update({"exp": expire})

    encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

    return encoded_jwt

def get_current_user(token: str = Header(..., description="Bearer token")):
    """Dependency: Get current user from token"""
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        username: str = payload.get("sub")

        if username is None:
            raise HTTPException(status_code=401)

        token_data = TokenData(username=username)

    except JWTError:
        raise HTTPException(status_code=401, detail="Invalid token")

```

```

    if token_data.username not in fake_users_db:
        raise HTTPException(status_code=401)

    user = fake_users_db[token_data.username]
    return User(**user)

# Login endpoint
@app.post("/login", response_model=Token)
async def login(username: str, password: str):
    """
    Login endpoint that returns JWT token

    Usage:
    POST /login
    {"username": "john", "password": "password123"}
    """
    user = authenticate_user(username, password)

    if not user:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid credentials"
        )

    access_token_expires = timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    access_token = create_access_token(
        data={"sub": user["username"]},
        expires_delta=access_token_expires
    )

    return {"access_token": access_token, "token_type": "bearer"}

# Protected endpoint
@app.get("/users/me", response_model=User)
async def read_users_me(current_user: User = Depends(get_current_user)):
    """Protected endpoint that requires JWT token"""
    return current_user

# Usage flow:
# 1. POST /login with username and password
# 2. Receive access_token
# 3. Use token: Authorization: Bearer <token>
# 4. Access protected endpoints

```

JWT Token Structure:


```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

# Configure CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "https://example.com",
        "https://www.example.com",
        "http://localhost:3000" # For development
    ],
    allow_credentials=True,
    allow_methods=["*"], # Allow all methods
    allow_headers=["*"], # Allow all headers
)

@app.get("/public")
def public_endpoint():
    return {"message": "This is public"}
```

CORS with Wildcards (Development Only)

python

```
from fastapi.middleware.cors import CORSMiddleware

# DANGER: Only for development!
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"], # Allow all origins
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

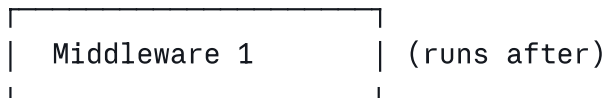
# SAFE: For production
app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "https://example.com",
        "https://api.example.com"
    ],
    allow_methods=["GET", "POST"],
    allow_headers=["authorization"],
    max_age=600, # Cache preflight for 10 minutes
)
```

4.4 Middleware

What is Middleware?

Middleware is code that runs before and after your route handlers.

Request comes in



Response goes out

Creating Custom Middleware

python

```
from fastapi import FastAPI, Request
import time

app = FastAPI()

@app.middleware("http")
async def add_process_time_header(request: Request, call_next):
    """Middleware that adds processing time to response"""

    # Code here runs BEFORE route handler
    start_time = time.time()

    # Call the route handler
    response = await call_next(request)

    # Code here runs AFTER route handler
    process_time = time.time() - start_time
    response.headers["X-Process-Time"] = str(process_time)

    return response

# Response will include:
# X-Process-Time: 0.123456
```

Request Logging Middleware

```
python

from fastapi import FastAPI, Request
import logging

app = FastAPI()
logger = logging.getLogger(__name__)

@app.middleware("http")
async def log_requests(request: Request, call_next):
    """Log all HTTP requests"""
    logger.info(f"{request.method} {request.url.path}")

    response = await call_next(request)

    logger.info(f"Status: {response.status_code}")

    return response
```


4.5 Background Tasks

Running Tasks in Background

```
python

from fastapi import FastAPI, BackgroundTasks
import time

app = FastAPI()

def write_notification(email: str, message: str = ""):
    """Background task"""
    # This runs in background, doesn't block response
    time.sleep(5) # Simulate work
    print(f"Email sent to {email}: {message}")

@app.post("/send-notification")
async def send_notification(email: str, bg_tasks: BackgroundTasks):
    """Add task to background queue"""
    bg_tasks.add_task(write_notification, email, message="Hello!")

    # Response sent immediately
    return {"message": "Notification queued"}

# Client receives response immediately
# Task runs in background after response is sent
```

Multiple Background Tasks

```
python
```

```

from fastapi import FastAPI, BackgroundTasks

app = FastAPI()

def task1():
    print("Task 1 running")

def task2():
    print("Task 2 running")

@app.post("/do-work")
async def do_work(bg_tasks: BackgroundTasks):
    bg_tasks.add_task(task1)
    bg_tasks.add_task(task2)

    return {"message": "Tasks queued"}

# Both tasks run in background

```

4.6 Rate Limiting

Simple Rate Limiting with SlowAPI

```

bash

pip install slowapi

```

```

python

from fastapi import FastAPI, Request
from slowapi import Limiter
from slowapi.util import get_remote_address

app = FastAPI()
limiter = Limiter(key_func=get_remote_address)

@app.get("/limited")
@limiter.limit("5/minute")
async def limited_endpoint(request: Request):
    """Allow 5 requests per minute"""
    return {"message": "Success"}

# After 5 requests in a minute:
# 429 Too Many Requests

```

4.7 DAYS 4-5 EXERCISES

Exercise 4.1: Secure API with JWT

Create an API with:

- User login endpoint that returns JWT
- Protected endpoints that require JWT
- Token expiration
- Password hashing

Exercise 4.2: Multi-tenant API

Create an API where:

- Dependency injection retrieves current user from token
- Users only see their own data
- Admin users see all data
- Role-based access control

Exercise 4.3: API with Logging and Monitoring

Add:

- Request/response logging middleware
 - Execution time tracking
 - Error logging
 - Request ID tracking for debugging
-

PART 5: DAYS 6-7 - PRODUCTION & REAL-WORLD

5.1 Database Integration with SQLAlchemy

ORM Basics

```
python
```

```

from sqlalchemy import Column, Integer, String, Float, Boolean, create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker

# Database URL
DATABASE_URL = "sqlite:///./test.db"

# Create engine
engine = create_engine(
    DATABASE_URL,
    connect_args={"check_same_thread": False} # SQLite only
)

# Create session factory
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

# Base class for models
Base = declarative_base()

# Define model
class Product(Base):
    __tablename__ = "products"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(50), index=True)
    description = Column(String(255))
    price = Column(Float)
    in_stock = Column(Boolean, default=True)

# Create tables
Base.metadata.create_all(bind=engine)

```

Connecting to FastAPI

```
python
```

```

from fastapi import FastAPI, Depends
from sqlalchemy.orm import Session

app = FastAPI()

def get_db():
    """Dependency: Provide database session"""
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

@app.post("/products")
def create_product(
    name: str,
    description: str,
    price: float,
    db: Session = Depends(get_db)
):
    """Create product in database"""
    product = Product(
        name=name,
        description=description,
        price=price
    )

    db.add(product)
    db.commit()
    db.refresh(product)

    return product

@app.get("/products/{product_id}")
def get_product(product_id: int, db: Session = Depends(get_db)):
    """Get product from database"""
    product = db.query(Product).filter(Product.id == product_id).first()

    if not product:
        raise HTTPException(status_code=404)

    return product

@app.get("/products")
def list_products(
    skip: int = 0,

```

```

        limit: int = 10,
        db: Session = Depends(get_db)
    ):
        """List products with pagination"""
        products = db.query(Product).offset(skip).limit(limit).all()
        return products

@app.put("/products/{product_id}")
def update_product(
    product_id: int,
    name: str = None,
    price: float = None,
    db: Session = Depends(get_db)
):
    """Update product"""
    product = db.query(Product).filter(Product.id == product_id).first()

    if not product:
        raise HTTPException(status_code=404)

    if name:
        product.name = name
    if price:
        product.price = price

    db.commit()
    db.refresh(product)

    return product

@app.delete("/products/{product_id}")
def delete_product(product_id: int, db: Session = Depends(get_db)):
    """Delete product"""
    product = db.query(Product).filter(Product.id == product_id).first()

    if not product:
        raise HTTPException(status_code=404)

    db.delete(product)
    db.commit()

    return {"deleted": True}

```

Relationships: One-to-Many

python

```

from sqlalchemy import ForeignKey, create_engine
from sqlalchemy.orm import relationship

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True)
    name = Column(String)

    # Relationship
    posts = relationship("Post", back_populates="author")

class Post(Base):
    __tablename__ = "posts"

    id = Column(Integer, primary_key=True)
    title = Column(String)
    user_id = Column(Integer, ForeignKey("users.id"))

    # Relationship
    author = relationship("User", back_populates="posts")

# Usage
user = db.query(User).first()
print(user.posts) # Get all posts by user
print(user.posts[0].author) # Get author of post

```

5.2 Environment Configuration

Using Environment Variables

```

bash

# .env file
DATABASE_URL=postgresql://user:password@localhost/dbname
SECRET_KEY=your-secret-key
DEBUG=true

```

```
python
```

```
from pydantic import BaseSettings

class Settings(BaseSettings):
    database_url: str = "sqlite:///./test.db"
    secret_key: str = "your-secret"
    debug: bool = False
    api_title: str = "My API"

    class Config:
        env_file = ".env"

settings = Settings()

# Usage
from sqlalchemy import create_engine
engine = create_engine(settings.database_url)

print(settings.api_title) # Loaded from .env or default
```

Settings per Environment

python


```
from pydantic import BaseSettings
from typing import Optional

class Settings(BaseSettings):
    env: str = "development"
    database_url: str
    secret_key: str
    debug: bool

    class Config:
        case_sensitive = True

# Load based on environment
import os
env = os.getenv("ENV", "development")

if env == "development":
    settings = Settings(
        database_url="sqlite:///./test.db",
        secret_key="dev-secret",
        debug=True
    )
else:
    settings = Settings(
        database_url=os.getenv("DATABASE_URL"),
        secret_key=os.getenv("SECRET_KEY"),
        debug=False
    )
```

5.3 Error Handling & Logging

Exception Handlers

python

```
from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse
from fastapi.exceptions import RequestValidationError
import logging

app = FastAPI()
logger = logging.getLogger(__name__)

# Handle validation errors
@app.exception_handler(RequestValidationError)
async def validation_exception_handler(request: Request, exc: RequestValidationError):
    """Custom handling of validation errors"""
    logger.error(f"Validation error on {request.url}: {exc}")

    return JSONResponse(
        status_code=422,
        content={
            "detail": "Validation error",
            "errors": exc.errors()
        }
    )

# Handle generic exceptions
@app.exception_handler(Exception)
async def generic_exception_handler(request: Request, exc: Exception):
    """Catch-all for unexpected errors"""
    logger.error(f"Unexpected error: {exc}", exc_info=True)

    return JSONResponse(
        status_code=500,
        content={"detail": "Internal server error"}
    )
```

Structured Logging

python

```

import logging
from logging.handlers import RotatingFileHandler

# Configure logging
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s'
)

logger = logging.getLogger(__name__)

# Add file handler
handler = RotatingFileHandler(
    "app.log",
    maxBytes=1000000, # 1MB
    backupCount=10
)
logger.addHandler(handler)

# Usage in FastAPI
@app.post("/items")
async def create_item(item: dict):
    try:
        logger.info(f"Creating item: {item}")
        # ... create item ...
        logger.info("Item created successfully")
        return item
    except Exception as e:
        logger.error(f"Error creating item: {e}", exc_info=True)
        raise

```

5.4 API Versioning

Versioning with Path Prefix

python

```

from fastapi import FastAPI

app = FastAPI()

# Version 1 endpoints
@app.get("/api/v1/users")
def list_users_v1():
    return {"users": [], "version": 1}

# Version 2 endpoints
@app.get("/api/v2/users")
def list_users_v2():
    return {
        "users": [],
        "version": 2,
        "total_count": 0 # New in v2
    }

```

Using APIRouter for Organization

```

python

from fastapi import APIRouter, FastAPI

app = FastAPI()

# Version 1 router
v1_router = APIRouter(prefix="/api/v1", tags=["V1"])

@v1_router.get("/users")
def list_users_v1():
    return {"users": []}

# Version 2 router
v2_router = APIRouter(prefix="/api/v2", tags=["V2"])

@v2_router.get("/users")
def list_users_v2():
    return {"users": [], "total": 0}

# Include routers
app.include_router(v1_router)
app.include_router(v2_router)

```

5.5 Testing

Unit Tests with pytest

```
bash
```

```
pip install pytest httpx
```

```
python
```

```
# test_api.py
from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

def test_read_items():
    """Test GET /items"""
    response = client.get("/items")
    assert response.status_code == 200
    assert "items" in response.json()

def test_create_item():
    """Test POST /items"""
    response = client.post(
        "/items",
        json={"name": "Test Item", "price": 99.99}
    )
    assert response.status_code == 201
    assert response.json()["name"] == "Test Item"

def test_get_item():
    """Test GET /items/{id}"""
    response = client.get("/items/1")
    assert response.status_code == 200

def test_delete_item():
    """Test DELETE /items/{id}"""
    response = client.delete("/items/1")
    assert response.status_code == 204
```

Testing with Dependencies

```
python
```

```

from fastapi.testclient import TestClient
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from main import app, get_db, Base, Product

# Use in-memory database for testing
SQLALCHEMY_TEST_DATABASE_URL = "sqlite:///./test.db"
engine = create_engine(SQLALCHEMY_TEST_DATABASE_URL)

Base.metadata.create_all(bind=engine)

TestingSessionLocal = sessionmaker(bind=engine)

def override_get_db():
    try:
        db = TestingSessionLocal()
        yield db
    finally:
        db.close()

app.dependency_overrides[get_db] = override_get_db

client = TestClient(app)

def test_create_product():
    response = client.post(
        "/products",
        json={"name": "Test", "price": 10.0}
    )
    assert response.status_code == 201

```

5.6 Docker Containerization

Dockerfile

dockerfile

```
FROM python:3.9-slim

WORKDIR /app

# Install dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy code
COPY . .

# Expose port
EXPOSE 8000

# Run application
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

requirements.txt

```
fastapi==0.104.1
uvicorn[standard]==0.24.0
sqlalchemy==2.0.23
pydantic==2.5.0
pydantic-settings==2.1.0
python-jose[cryptography]==3.3.0
passlib[bcrypt]==1.7.4
python-multipart==0.0.6
```

docker-compose.yml

```
yaml
```

```
version: '3.8'

services:
  api:
    build: .
    ports:
      - "8000:8000"
    environment:
      - DATABASE_URL=postgresql://user:password@db:5432/mydb
      - SECRET_KEY=your-secret-key
    depends_on:
      - db

  db:
    image: postgres:15
    environment:
      - POSTGRES_USER=user
      - POSTGRES_PASSWORD=password
      - POSTGRES_DB=mydb
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data:
```

Run with Docker

```
bash

# Build image
docker build -t my-api .

# Run container
docker run -p 8000:8000 my-api

# Or with docker-compose
docker-compose up
```

5.7 Deployment Options

Deployment Checklist

- ☐ Use production database (PostgreSQL, MySQL)

- ☐ Set DEBUG=False
- ☐ Use strong SECRET_KEY
- ☐ Enable HTTPS
- ☐ Configure CORS properly
- ☐ Add logging
- ☐ Set up error monitoring (Sentry)
- ☐ Use environment variables for secrets
- ☐ Run tests before deployment
- ☐ Set up CI/CD pipeline
- ☐ Monitor application performance

Heroku Deployment

bash

1. Create requirements.txt

```
pip freeze > requirements.txt
```

2. Create Procfile

```
echo "web: uvicorn main:app --host=0.0.0.0 --port=${PORT:-8000}" > Procfile
```

3. Create app.json

```
echo '{
  "name": "My FastAPI App",
  "description": "A FastAPI application"
}' > app.json
```

4. Deploy

```
heroku create my-app
```

```
git push heroku main
```

AWS Lambda with Mangum

bash

```
pip install mangum
```

python

```
# main.py
from fastapi import FastAPI
from mangum import Mangum

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Hello from Lambda"}

# Lambda handler
handler = Mangum(app)
```

COMMON MISTAKES & MISCONCEPTIONS

Mistake 1: Mixing Async/Sync Incorrectly

```
python

# ❌ WRONG: Using sync function with async
import time

@app.get("/items")
async def read_items():
    time.sleep(5) # Blocking! Defeats async purpose
    return {"items": []}

# ✅ CORRECT: Use async operations
import asyncio

@app.get("/items")
async def read_items():
    await asyncio.sleep(5) # Non-blocking
    return {"items": []}

# ✅ OR: Use sync if no async version available
@app.get("/items")
def read_items(): # Not async
    time.sleep(5)
    return {"items": []}
```

Mistake 2: Validating Query Parameters as Path Parameters

```
python
```

```
# ❌ WRONG
@app.get("/users/{user_id}")
def get_user(user_id): # No type hint
    return {"user_id": user_id}

# ✅ CORRECT
@app.get("/users/{user_id}")
def get_user(user_id: int): # Automatic validation
    return {"user_id": user_id}
```

Mistake 3: Not Using Response Models

```
python

# ❌ WRONG: Returning raw database object exposes everything
@app.get("/users/{user_id}")
def get_user(user_id: int, db: Session = Depends(get_db)):
    user = db.query(User).filter(User.id == user_id).first()
    return user # Exposes password_hash, internal fields!

# ✅ CORRECT: Use response model to control output
class UserResponse(BaseModel):
    id: int
    name: str
    email: str

@app.get("/users/{user_id}", response_model=UserResponse)
def get_user(user_id: int, db: Session = Depends(get_db)):
    user = db.query(User).filter(User.id == user_id).first()
    return user # Only returns specified fields
```

Mistake 4: Hardcoding Configuration

```
python
```

```
# ❌ WRONG
DATABASE_URL = "postgresql://user:password@localhost/db"
SECRET_KEY = "hardcoded-secret"
DEBUG = True

# ✅ CORRECT
import os
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    database_url: str = os.getenv("DATABASE_URL", "sqlite:///./test.db")
    secret_key: str = os.getenv("SECRET_KEY")
    debug: bool = os.getenv("DEBUG", "false").lower() == "true"

settings = Settings()
```

Mistake 5: Not Handling Database Sessions Properly

```
python

# ❌ WRONG: Database connection not closed
@app.get("/items")
def read_items():
    db = SessionLocal()
    items = db.query(Item).all()
    return items # Session never closed!

# ✅ CORRECT: Use dependency injection
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close() # Always closes

@app.get("/items")
def read_items(db: Session = Depends(get_db)):
    items = db.query(Item).all()
    return items # Session auto-closed
```

Mistake 6: Not Using Status Codes Correctly

```
python
```

```
# ❌ WRONG
@app.post("/items")
def create_item(item: dict):
    # Implicitly returns 200 OK
    return item

# ✅ CORRECT: Use appropriate status code
from fastapi import status

@app.post("/items", status_code=status.HTTP_201_CREATED)
def create_item(item: dict):
    # Returns 201 Created
    return item

@app.delete("/items/{id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_item(id: int):
    # Returns 204 No Content
    return None
```

Mistake 7: Storing Sensitive Data in Logs

```
python

# ❌ WRONG
import logging
logger = logging.getLogger(__name__)

@app.post("/login")
def login(username: str, password: str):
    logger.info(f"Login attempt: {username} {password}") # NEVER log passwords!
    return {"token": "..."}

# ✅ CORRECT
@app.post("/login")
def login(username: str, password: str):
    logger.info(f"Login attempt for user: {username}")
    # Verify password without logging it
    return {"token": "..."}

```

ADVANCED CONCEPTS

1. Dependency Injection with Scopes

FastAPI's dependency system supports scopes:

python

```
from fastapi import FastAPI, Depends

app = FastAPI()

class DatabaseConnection:
    def __init__(self):
        self.id = str(uuid4())
        print(f"Creating DB connection {self.id}")

    def __del__(self):
        print(f"Closing DB connection {self.id}")

# Function scope (default): Created fresh for each route
def get_db_function():
    return DatabaseConnection()

# Request scope: Cached within single request
async def get_db_request():
    return DatabaseConnection()

@app.get("/items")
async def read_items(db1 = Depends(get_db_function), db2 = Depends(get_db_function))
    # db1 and db2 are DIFFERENT instances
    # Both have different IDs
    return {}
```

2. Streaming Responses

python

```

from fastapi import FastAPI
from fastapi.responses import StreamingResponse
import io

app = FastAPI()

@app.get("/stream")
async def stream_large_file():
    """Stream a large file without loading it all in memory"""
    def iterfile():
        with open("large_file.txt") as file:
            yield from file

    return StreamingResponse(
        iterfile(),
        media_type="text/plain"
    )

@app.get("/csv-export")
async def export_csv():
    """Stream CSV data"""
    def generate():
        yield "id,name,email\n"
        for user in users:
            yield f"{user.id},{user.name},{user.email}\n"

    return StreamingResponse(
        generate(),
        media_type="text/csv",
        headers={"Content-Disposition": "attachment; filename=users.csv"}
    )

```

3. WebSocket Support

python

```
from fastapi import FastAPI, WebSocket

app = FastAPI()

@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()

    try:
        while True:
            data = await websocket.receive_text()
            await websocket.send_text(f"Echo: {data}")
    except Exception:
        await websocket.close()

# Client-side (JavaScript):
# const ws = new WebSocket("ws://localhost:8000/ws");
# ws.send("Hello!");
```

4. OpenAPI Customization

```
python
```



```
from fastapi import FastAPI

app = FastAPI(
    title="Advanced API",
    description="A detailed API description",
    version="2.0.0",
    openapi_url="/api/v1/openapi.json",
    docs_url="/api/v1/docs",
    redoc_url="/api/v1/redoc"
)

@app.get("/items", tags=["Items"], summary="List items")
def list_items():
    """
    ## Get all items

    This endpoint returns a list of all available items.

    ### Parameters
    - **skip**: Number of items to skip
    - **limit**: Maximum items to return

    ### Returns
    List of Item objects
    """
    return []
```

5. GraphQL with FastAPI

bash

```
pip install strawberry-graphql
```

python

```

import strawberry
from fastapi import FastAPI
from strawberry.asgi import GraphQL

@strawberry.type
class User:
    id: int
    name: str
    email: str

@strawberry.type
class Query:
    @strawberry.field
    def user(self, id: int) -> User:
        return User(id=1, name="John", email="john@example.com")

schema = strawberry.Schema(query=Query)

app = FastAPI()
app.add_route("/graphql", GraphQL(schema))

```

6. Rate Limiting Strategies

```

python

from slowapi import Limiter, _rate_limit_exceeded_handler
from slowapi.util import get_remote_address
from slowapi.errors import RateLimitExceeded

limiter = Limiter(key_func=get_remote_address)
app = FastAPI()
app.state.limiter = limiter
app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler)

# Different limits for different endpoints
@app.get("/public")
@limiter.limit("100/hour")
async def public_endpoint(request: Request):
    return {"status": "ok"}

@app.post("/expensive")
@limiter.limit("10/hour") # Stricter limit
async def expensive_operation(request: Request):
    return {"status": "ok"}

```

RESOURCES & REFERENCES

Official Documentation

- **FastAPI Documentation:** <https://fastapi.tiangolo.com>
- **Pydantic Documentation:** <https://docs.pydantic.dev>
- **Uvicorn Documentation:** <https://www.uvicorn.org>
- **SQLAlchemy Documentation:** <https://docs.sqlalchemy.org>

Recommended Tools & Libraries

API Development:

- **HTTPIe:** User-friendly HTTP client (alternative to curl)
- **Postman:** API testing and collaboration
- **Insomnia:** REST/GraphQL client

Database:

- **pgAdmin:** PostgreSQL management
- **MongoDB Compass:** MongoDB GUI
- **DBeaver:** Universal database tool

DevOps:

- **Docker:** Containerization
- **Docker Compose:** Multi-container orchestration
- **GitHub Actions:** CI/CD automation

Monitoring & Logging:

- **Sentry:** Error tracking
- **LogRocket:** Session replay and logging
- **Datadog:** Infrastructure monitoring

Useful FastAPI Extensions

fastapi-pagination	# Easy pagination
python-multipart	# Form data handling
sqlmodel	# SQLAlchemy + Pydantic combo
fastapi-cors	# CORS handling

```
fastapi-jwt-auth          # JWT authentication
fastapi-cache2            # Caching decorator
```

Communities & Forums

- **FastAPI Discord:** <https://discord.gg/VQjSZaeJmf>
- **Stack Overflow:** Tag: `fastapi`
- **GitHub Discussions:** FastAPI repository

Learning Resources

- **Real Python:** FastAPI tutorials
- **Full Stack Python:** Web framework comparisons
- **Awesome FastAPI:** Curated list of resources

QUICK REVISION SHEET

Installation

```
bash

pip install fastapi uvicorn sqlalchemy pydantic python-jose
```

Project Structure

```
my_api/
├── main.py          # Main application file
├── models.py        # Pydantic and SQLAlchemy models
├── database.py       # Database configuration
├── schemas.py       # Request/response schemas
├── crud.py          # Database operations
├── dependencies.py   # Dependency functions
├── requirements.txt  # Package dependencies
├── .env             # Environment variables
├── tests/           # Test files
└── Dockerfile       # Containerization
```

Basic API Template

```
python
```

```
from fastapi import FastAPI, Depends, HTTPException, status
from fastapi.middleware.cors import CORSMiddleware
from sqlalchemy.orm import Session
from pydantic import BaseModel
import os

# Config
class Settings(BaseSettings):
    database_url: str = os.getenv("DATABASE_URL")
    secret_key: str = os.getenv("SECRET_KEY")

settings = Settings()

# Database
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

engine = create_engine(settings.database_url)
SessionLocal = sessionmaker(bind=engine)

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

# Models
from sqlalchemy.ext.declarative import declarative_base
Base = declarative_base()

class UserModel(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True)
    name = Column(String)

Base.metadata.create_all(bind=engine)

# Schemas
class UserCreate(BaseModel):
    name: str

class UserResponse(BaseModel):
    id: int
    name: str
```

```

# App
app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"],
    allow_methods=["*"],
    allow_headers=["*"],
)

# Routes
@app.post("/users", response_model=UserResponse, status_code=201)
def create_user(user: UserCreate, db: Session = Depends(get_db)):
    db_user = UserModel(name=user.name)
    db.add(db_user)
    db.commit()
    db.refresh(db_user)
    return db_user

@app.get("/users/{user_id}", response_model=UserResponse)
def get_user(user_id: int, db: Session = Depends(get_db)):
    user = db.query(UserModel).filter(UserModel.id == user_id).first()
    if not user:
        raise HTTPException(status_code=404)
    return user

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)

```

HTTP Methods Cheat Sheet

Method	Purpose	Example	Status Code
GET	Retrieve	Get users	200 OK
POST	Create	Create user	201 Created
PUT	Replace	Replace user	200 OK
PATCH	Update	Update email	200 OK
DELETE	Remove	Delete user	204 No Content

Status Codes Quick Reference

Code	Meaning	Use Case
200	OK	Successful GET/PUT/PATCH
201	Created	Successful POST
204	No Content	DELETE successful
400	Bad Request	Invalid input
401	Unauthorized	Missing/invalid auth
403	Forbidden	Authenticated but no access
404	Not Found	Resource doesn't exist
422	Validation Error	Validation failed
500	Server Error	Unexpected error

Common Decorators

```
python

@app.get(path, response_model=None)
@app.post(path, status_code=201)
@app.put(path)
@app.patch(path)
@app.delete(path, status_code=204)

# With dependencies
@app.get("/secure")
@limiter.limit("10/minute")
async def endpoint(...):
```

Type Hints Quick Reference

```
python
```

```
from typing import List, Optional, Dict

parameter: str # Required string
parameter: str = "default" # Optional with default
parameter: Optional[str] = None # Explicitly optional
parameter: int # Required int
parameter: float # Required float
parameter: bool # Required bool
parameter: List[str] # List of strings
parameter: Dict[str, int] # Dictionary
parameter: List[UserModel] # List of models
```

Running & Testing

```
bash

# Development
uvicorn main:app --reload

# Production
uvicorn main:app --host 0.0.0.0 --port 8000 --workers 4

# Testing
pytest test_api.py
pytest --cov=.

# Docker
docker build -t api .
docker run -p 8000:8000 api
```

INTERVIEW QUESTIONS

Beginner Level

1. What is FastAPI and why would you use it? *Answer:* FastAPI is a modern Python web framework for building APIs with automatic data validation, serialization, and documentation. It's fast (2-3x faster than Flask), easy to learn, and includes built-in features like Swagger UI documentation.

2. What's the difference between path parameters and query parameters? *Answer:* Path parameters are part of the URL path (`/users/123`), while query parameters come after a `?` in the URL (`/users?skip=0&limit=10`). Path parameters are typically required for identifying specific resources, while query parameters are optional filters.

3. What is Pydantic and how does it help in FastAPI? Answer: Pydantic is a Python library that enforces type hints at runtime. In FastAPI, it validates request data, serializes responses, and automatically generates API documentation from model definitions.

4. How do you create a GET endpoint in FastAPI?

```
python

@app.get("/items/{item_id}")
def read_item(item_id: int):
    return {"item_id": item_id}
```

5. What's the purpose of the `response_model` parameter? Answer: `response_model` specifies which fields should be included in the response and validates that the data matches the model. It also controls which sensitive fields are exposed (e.g., hiding password hashes).

Intermediate Level

6. Explain FastAPI's dependency injection system. Answer: FastAPI's `Depends()` allows you to declare reusable pieces of logic that can be injected into route handlers. This enables code reuse, testing, and separation of concerns. Dependencies can be nested and cached within a request.

7. How would you handle authentication in FastAPI? Answer: FastAPI provides built-in security schemes like HTTPBasic and HTTPBearer. For JWT, you decode tokens in a dependency function that validates the user and injects it into the route handler.

8. What's the difference between `.commit()` and `.refresh()` in SQLAlchemy? Answer: `.commit()` persists changes to the database, while `.refresh()` fetches the latest data from the database for an object (useful to get auto-generated IDs after insert).

9. How do you handle validation errors in FastAPI? Answer: FastAPI automatically returns 422 Validation Error for validation failures. You can customize responses using custom validators in Pydantic models or exception handlers in FastAPI.

10. Explain the request-response cycle in FastAPI. Answer: 1) Request arrives → 2) Uvicorn parses HTTP → 3) FastAPI routes to handler → 4) Dependencies resolved → 5) Parameters validated → 6) Handler executes → 7) Response serialized → 8) Response returned.

Advanced Level

11. How would you implement rate limiting in FastAPI? Answer: Use the `slowapi` library with a limiter decorator:

```
python
```

```
@app.get("/items")
@limiter.limit("10/minute")
async def get_items(request: Request):
    return []
```

12. What are middleware and how do you create custom middleware in FastAPI? *Answer:* Middleware runs before and after route handlers. Create with:

```
python

@app.middleware("http")
async def custom_middleware(request: Request, call_next):
    response = await call_next(request)
    response.headers["X-Custom"] = "value"
    return response
```

13. How do you implement JWT authentication with expiration? *Answer:* Create JWT tokens with `jwt.encode()` including an `exp` claim. Decode and validate in a dependency, raising `HTTPException` if expired or invalid.

14. Explain `async/await` in FastAPI and when to use it. *Answer:* Use `async def` for I/O-bound operations (database, API calls) to avoid blocking. Use regular `def` for CPU-bound operations. FastAPI runs `async` routes concurrently using an event loop.

15. How would you structure a large FastAPI application? *Answer:*

```
api/
├─ main.py
├─ models/
│   └─ user.py
│   └─ product.py
├─ schemas/
│   └─ user.py
│   └─ product.py
├─ crud/
│   └─ user.py
│   └─ product.py
├─ routes/
│   └─ users.py
│   └─ products.py
├─ dependencies.py
├─ database.py
└─ config.py
```

16. What's the difference between `PUT` and `PATCH`? *Answer:* `PUT` replaces the entire resource (idempotent), while `PATCH` applies partial updates. `PUT` requires all fields, `PATCH` only changed

fields.

17. How do you test a FastAPI application? *Answer:* Use TestClient and pytest:

```
python

from fastapi.testclient import TestClient
client = TestClient(app)
response = client.get("/items")
assert response.status_code == 200
```

18. Explain CORS and how to configure it in FastAPI. *Answer:* CORS (Cross-Origin Resource Sharing) allows browsers to make requests to different domains. Configure with:

```
python

app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://example.com"],
    allow_methods=["GET", "POST"],
)
```

19. How would you optimize FastAPI for high traffic? *Answer:*

- Use async/await properly
- Add caching (Redis, FastAPI-Cache)
- Implement rate limiting
- Use connection pooling for databases
- Containerize and scale with multiple workers
- Monitor with tools like Sentry

20. What are some common mistakes when using FastAPI? *Answer:*

- Mixing async and blocking operations
- Not using response_model (security/privacy issue)
- Not handling database sessions properly
- Hardcoding configuration
- Not validating user input
- Using sync dependencies when async is needed

System Design Questions

21. Design a scalable user authentication system in FastAPI. *Answer:*

- Implement JWT with refresh tokens

- Store refresh tokens in Redis
- Hash passwords with bcrypt
- Rate limit login attempts
- Use HTTPS only
- Implement token revocation
- Add 2FA support

22. How would you design a multi-tenant API? *Answer:*

- Extract tenant ID from header/subdomain
 - Use dependency to validate tenant access
 - Filter database queries by tenant
 - Isolate databases per tenant (optional)
 - Implement row-level security
-

Document Summary

This comprehensive guide covers:

-  7-day structured learning plan
-  Beginner to advanced FastAPI concepts
-  Real-world practical examples
-  Database integration and ORM
-  Authentication and security
-  Testing and deployment
-  Common mistakes to avoid
-  Interview preparation
-  Quick reference sheet

Time Investment: ~28-35 hours (4-5 hours per day) **Outcome:** Production-ready FastAPI developer

Version: 1.0

Last Updated: 2025

Status: Print-Ready ✓